

SOC HOME LAB

Security Operations Center Simulation

Attack Simulation & Detection Report

Phase 1 (No WAF) to Phase 2 (ModSecurity + OWASP CRS)

Date	April 26-27, 2026
Classification	Confidential - Lab Use Only
Network	192.168.75.0/24 VMware Workstation
Target Application	OWASP Juice Shop v19.2.1
SIEM Platform	Wazuh v4.14.5
WAF	ModSecurity v3.0.12 + OWASP CRS 3.3.5
Attacker Platform	Kali Linux 2024 (192.168.75.128)
Document Version	v1.0 Final

Contents

1. Executive Summary	3
1.1 Key Results Summary	3
2. Lab Environment & Network Topology	5
2.1 Hypervisor & Network Configuration	5
2.2 Virtual Machines	5
2.3 Software Stack	5
Attacker VM - Kali Linux (192.168.75.128)	5
SIEM VM - ubuntu-server (192.168.75.139)	5
Victim VM - ubuntu-victim (192.168.75.132)	6
2.4 Network Traffic Flow	6
2.5 Logging Pipeline	6
3. Phase 1 - Attack Simulation Without WAF	8
3.1 Attack 1 - Reflected Cross-Site Scripting (XSS)	8
Description	8

Command (Kali)	8
Server Response	8
Nginx Log Entry	8
Analysis	8
3.2 Attack 2 - SQL Injection Authentication Bypass	8
Description	8
Command (Kali)	8
Server Response - Authentication Bypass Successful	8
Analysis	8
3.3 Attack 3 - Active Reconnaissance with Nmap NSE	9
Command (Kali)	9
Representative Log Entries	9
Analysis	9
3.4 Attack 4 - Brute Force with Hydra	9
Command (Kali)	9
Hydra Output	9
4. WAF Deployment - ModSecurity + OWASP CRS	10
4.1 Installation	10
Rules Loaded Confirmation	10
4.2 Configuration Files	10
/etc/modsecurity/modsecurity.conf	10
/etc/modsecurity/main.conf	10
/etc/nginx/sites-available/juiceshop	10
4.3 OWASP CRS Anomaly Scoring	10
5. Phase 2 - Attack Simulation With WAF Active	11
5.1 XSS - Blocked	11
ModSecurity Error Log	11
5.2 SQL Injection - Blocked	11
5.3 Hydra Brute Force - Blocked	11
Nginx Access Log	11
6. Phase 1 vs Phase 2 - Full Comparison	12
7. Wazuh SIEM - Detection & Alerting	13
7.1 Agent Configuration	13
7.2 Log Sources Monitored	13
7.3 Built-in Wazuh Rules That Fired	13
7.4 Custom Detection Rules Created	14
Sample Custom Rule Code - Hydra Detection (Rule 100010)	14
8. MITRE ATT&CK Framework Coverage	15

9. Findings & Recommendations	16
9.1 Findings.....	16
9.2 Recommendations.....	16
10. Next Steps - Lab Roadmap	17
Phase 3 - Advanced Attack Techniques	17
Phase 4 - Detection Engineering	17
Phase 5 - Incident Response Simulation	17
Phase 6 - Compliance & Hardening.....	17
Appendix A - Configuration Reference	18
A.1 Systemd Services - Victim VM	18
A.2 Wazuh Manager Services - SIEM VM	18
A.3 Snapshot History - Victim VM.....	18
A.4 Wazuh Agent Log Sources (ossec.conf).....	18

1. Executive Summary

This report documents the design, deployment, and operational results of a Security Operations Center (SOC) home lab built on native software inside VMware Workstation VMs. The lab simulates a realistic enterprise threat environment covering active exploitation, detection, alerting, and response using industry-standard open-source tooling.

The exercise ran in two phases. Phase 1 had Kali Linux attacking OWASP Juice Shop with no application-layer defenses. Phase 2 deployed ModSecurity + OWASP CRS as a WAF and repeated the identical attacks. A Wazuh agent forwarded all logs to a centralized SIEM where custom MITRE ATT&CK-mapped rules generated real-time alerts.

1.1 Key Results Summary

Attack	Phase 1 - No WAF	Phase 2 - WAF Active
XSS Reflected	HTTP 200 - PASSED (not blocked)	HTTP 403 - BLOCKED
SQL Injection Auth Bypass	HTTP 200 - Admin JWT obtained	HTTP 403 - BLOCKED
Hydra Brute Force (102 attempts)	Requests reached application	HTTP 403 x24 - ALL BLOCKED
Nmap NSE Reconnaissance	Full server fingerprint completed	Logged - Custom Rule 100012
Admin Endpoint Access	HTTP 200 + full config returned	Detected - Custom Rule 100014

Attack	Phase 1 - No WAF	Phase 2 - WAF Active
SQLMap Scanner	Application crashed (DoS)	Detected - Custom Rule 100011
SIEM Alerts Generated	2 generic web alerts	12+ categorized + MITRE mapped
Custom Detection Rules Active	0	6 rules (100010-100015)

2. Lab Environment & Network Topology

2.1 Hypervisor & Network Configuration

Parameter	Value
Hypervisor	VMware Workstation
Network Segment	192.168.75.0/24
Network Mode	Host-only / NAT
NTP Server	ntp.ubuntu.com (stratum 2)
Timezone (all VMs)	America/Costa_Rica (CST, UTC-6)
Clock Sync (SIEM)	+58 microseconds offset
Clock Sync (Victim)	-5.079 milliseconds offset

2.2 Virtual Machines

VM Name	Role	IP Address	OS	CPU / RAM
Kali Linux	Attacker / Red Team	192.168.75.128	Kali Linux 2024	2c / 2GB
ubuntu-server	Wazuh SIEM Stack	192.168.75.139	Ubuntu 24.04.4 LTS	2c / 4GB
ubuntu-victim	Victim / Blue Team	192.168.75.132	Ubuntu 24.04.4 LTS	2c / 2GB

2.3 Software Stack

Attacker VM - Kali Linux (192.168.75.128)

Tool	Version	Purpose
Nmap	7.94	Network discovery and port scanning with NSE scripts
Hydra	9.6	Credential brute-force attacks against login endpoints
SQLMap	1.10 stable	Automated SQL injection detection and exploitation
Burp Suite	Community	Web proxy for manual request interception
curl	8.18.0	HTTP request crafting for manual attack tests

SIEM VM - ubuntu-server (192.168.75.139)

Component	Version	Role
Wazuh Manager	4.14.5	Central log analysis, correlation, and rule engine


```
|  
[Wazuh Manager: decode + rules]  
|  
[Wazuh Indexer: OpenSearch]  
|  
[Dashboard :443: Alerts + MITRE]
```

3. Phase 1 - Attack Simulation Without WAF

In Phase 1, Nginx was a plain reverse proxy with no ModSecurity. All HTTP traffic reached Juice Shop directly. This phase establishes the baseline of what attacks succeed and what telemetry is generated without application-layer defenses.

3.1 Attack 1 - Reflected Cross-Site Scripting (XSS)

Description

A reflected XSS payload was injected into the product search parameter. In a real scenario this allows stealing session cookies, redirecting victims, or executing arbitrary JavaScript in any users browser.

Command (Kali)

```
curl -s "http://192.168.75.132/rest/products/search?q=<script>alert(1)</script>"
```

Server Response

```
HTTP/1.1 200 OK
{"status":"success","data":[]}
```

Nginx Log Entry

```
192.168.75.128 - - [26/Apr/2026:23:18:16 +0000] "GET
/rest/products/search?q=<script>alert(1)</script> HTTP/1.1" 200 30
```

Analysis

- Result: HTTP 200 OK - payload passed through without filtering or blocking
- CVSS v3.1: 6.1 (Medium) - Reflected XSS
- MITRE ATT&CK: T1059.007 - Command and Scripting Interpreter: JavaScript
- Wazuh Rule Fired: 31106 - A web attack returned code 200 (success)

3.2 Attack 2 - SQL Injection Authentication Bypass

Description

The classic SQLi payload ' OR 1=1-- terminates the SQL string and appends a tautology, causing the database to return the admin account regardless of password supplied.

Command (Kali)

```
curl -s -X POST http://192.168.75.132/rest/user/login \
-H "Content-Type: application/json" \
-d '{"email":"\' OR 1=1--","password":"x"}'
```

Server Response - Authentication Bypass Successful

```
HTTP/1.1 200 OK
{ "authentication": {
  "token": "eyJ0eXAiOiJKV1Qi... (valid admin JWT)",
  "umail": "admin@juice-sh.op",
  "bid": 1 } }
```

Analysis

- Result: CRITICAL - Full authentication bypass achieved
- Admin JWT token obtained without valid credentials
- Backend database: SQLite (confirmed by SQLMap)
- CVSS v3.1: 9.8 (Critical) - SQL Injection with privilege escalation
- MITRE ATT&CK: T1190 - Exploit Public-Facing Application

3.3 Attack 3 - Active Reconnaissance with Nmap NSE

Command (Kali)

```
nmap -sV -sC -p 80 192.168.75.132
```

Representative Log Entries

Timestamp (UTC)	Request	Code	Significance
23:20:33	GET /robots.txt	200	Hidden route discovery
23:20:34	GET /.git/HEAD	200	Git repository exposure check
23:20:34	OPTIONS /	204	HTTP method enumeration
23:20:34	PROPFIND /	200	WebDAV detection attempt
23:20:34	POST /sdk	200	SDK endpoint probing
23:20:34	GET /nmaplowercheck...	200	Nmap fingerprint beacon

Analysis

- Result: Complete server fingerprinting achieved in under 1 second
- /.git/HEAD returned 200 - potential source code exposure
- User-Agent 'Nmap Scripting Engine' visible in all entries - easily detectable
- MITRE ATT&CK: T1595 - Active Scanning / T1046 - Network Service Discovery

3.4 Attack 4 - Brute Force with Hydra

Command (Kali)

```
hydra -l admin@juice-sh.op -P /tmp/passwords_small.txt \
  192.168.75.132 http-post-form \
  "/rest/user/login:email=^USER^&password=^PASS^:HTTP/1.1 401" \
  -t 5 -v
```

Hydra Output

```
102 login attempts executed
1 of 1 target completed, 0 valid password found (WAF blocked all)
```

- 102 password attempts sent with no rate limiting or lockout in Phase 1
- MITRE ATT&CK: T1110 - Brute Force

4. WAF Deployment - ModSecurity + OWASP CRS

4.1 Installation

```
apt install -y libnginx-mod-http-modsecurity modsecurity-crs libmodsecurity3t64
```

Rules Loaded Confirmation

```
ModSecurity-nginx v1.0.3 (rules loaded inline/local/remote: 0/921/0)
```

4.2 Configuration Files

/etc/modsecurity/modsecurity.conf

```
SecRuleEngine On # Active blocking mode (was DetectionOnly)
```

/etc/modsecurity/main.conf

```
Include /etc/modsecurity/modsecurity.conf
Include /etc/modsecurity/crs/crs-setup.conf
Include /usr/share/modsecurity-crs/rules/*.conf
```

/etc/nginx/sites-available/juiceshop

```
server {
    listen 80; server_name _;
    modsecurity on;
    modsecurity_rules_file /etc/modsecurity/main.conf;
    access_log /var/log/nginx/juiceshop_access.log;
    error_log /var/log/nginx/juiceshop_error.log;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

4.3 OWASP CRS Anomaly Scoring

The CRS uses anomaly scoring: each suspicious pattern matched adds points. When the total exceeds 5, the request is blocked with HTTP 403.

Attack Type	Anomaly Score	Threshold	Action
XSS <script>alert(1)	18	5	BLOCKED - HTTP 403
SQLi ' OR 1=1--	8	5	BLOCKED - HTTP 403
Hydra brute force POST	8	5	BLOCKED - HTTP 403
Normal GET /	0	5	ALLOWED - HTTP 200

5. Phase 2 - Attack Simulation With WAF Active

5.1 XSS - Blocked

```
curl -v "http://192.168.75.132/rest/products/search?q=<script>alert(1)</script>"
HTTP/1.1 403 Forbidden
```

ModSecurity Error Log

```
[error] ModSecurity: Access denied with code 403 (phase 2)
TX:ANOMALY_SCORE Value: 18 (threshold: 5)
Rule: REQUEST-949-BLOCKING-EVALUATION.conf ID: 949110
URI: /rest/products/search Client: 192.168.75.128
```

- XSS payload blocked before reaching Juice Shop
- Wazuh Rule 31333 fired: ModSecurity rejected a query
- Custom Wazuh Rule 100015 fired: XSS attempt detected in request

5.2 SQL Injection - Blocked

```
HTTP/1.1 403 Forbidden -- Admin JWT: NOT obtained
```

- Authentication bypass completely prevented
- ModSecurity anomaly score: 8 (threshold: 5) - SQLi pattern matched
- Wazuh Rule 31333 fired

5.3 Hydra Brute Force - Blocked

Nginx Access Log

```
192.168.75.128 [00:17:58] GET /rest/user/login HTTP/1.0 403 (Hydra)
192.168.75.128 [00:17:58] POST /rest/user/login HTTP/1.0 403 (Hydra)
... 24 total requests blocked in the same second
Hydra result: 0 valid passwords found
```

- 24 brute force attempts blocked in under 1 second
 - ModSecurity detected HTTP/1.0 + Hydra User-Agent pattern
 - Custom Wazuh Rule 100010 fired: Hydra brute force tool detected
-

6. Phase 1 vs Phase 2 - Full Comparison

Attack Vector	Tool	Phase 1	Phase 2	MITRE	Severity
Reflected XSS	curl	200 - passed	403 - BLOCKED	T1059.007	Medium
SQLi Auth Bypass	curl	200 + Admin JWT	403 - BLOCKED	T1190	CRITICAL
Brute Force	Hydra	Reached application	403 x24 blocked	T1110	High
Recon Scan	Nmap NSE	Full fingerprint	Logged Rule 100012	T1595	Info
Admin Endpoint	curl	200 + full config	Detected Rule 100014	T1078	High
Path Traversal	curl	200 (SPA handles)	200 (SPA handles)	T1083	Low
SQLMap Scanner	sqlmap	App crash (DoS)	Detected Rule 100011	T1190	High

7. Wazuh SIEM - Detection & Alerting

7.1 Agent Configuration

Parameter	Value
Agent Name	juiceshop-victim
Agent ID	002
Agent IP	192.168.75.132
Manager IP	192.168.75.139
Protocol	TCP/1514 (AES encrypted)
OS	Ubuntu 24.04.4 LTS
Wazuh Version	v4.14.5
Registration Date	Apr 26, 2026 @ 22:01:12 CST

7.2 Log Sources Monitored

Log File	Format	Content
/var/log/nginx/juiceshop_access.log	apache	All HTTP requests through Nginx
/var/log/nginx/juiceshop_error.log	apache	ModSecurity blocks and Nginx errors
/var/log/auth.log	syslog	SSH logins, sudo usage, PAM events
/var/log/dpkg.log	syslog	Package install and removal events
journald	journald	Systemd services, kernel messages
/var/ossec/logs/active-responses.log	syslog	Wazuh active response actions

7.3 Built-in Wazuh Rules That Fired

Rule ID	Level	Description	MITRE
31333	7	ModSecurity rejected a query	T1083
31106	6	A web attack returned code 200 (success)	T1190
31105	6	XSS (Cross Site Scripting) attempt	T1059.007
5402	3	Successful sudo to ROOT executed	T1548.003
5501	3	PAM: Login session opened	T1078
506	3	Wazuh agent stopped	T1562.001

7.4 Custom Detection Rules Created

Rule ID	Level	Description	MITRE	Trigger
100010	10	Hydra brute force tool detected	T1110	User-Agent: Hydra
100011	10	SQLMap scanner detected	T1190	User-Agent: sqlmap
100012	8	Nmap NSE scanner detected	T1595	User-Agent: Nmap Scripting Engine
100013	12	Multiple WAF blocks from same IP (10 in 60s)	T1190	if_matched_sid 31333 x10
100014	10	Juice Shop admin endpoint accessed	T1078	URL: /rest/admin
100015	10	XSS attempt detected in request	T1059.007	Pattern: script>, onerror=

Sample Custom Rule Code - Hydra Detection (Rule 100010)

```
<rule id="100010" level="10">
  <if_group>web</if_group>
  <match>Hydra</match>
  <description>Hydra brute force tool detected</description>
  <mitre><id>T1110</id></mitre>
</rule>
```

8. MITRE ATT&CK Framework Coverage

Technique	Name	Tactic	Observed In Lab	Detection Method
T1595	Active Scanning	Reconnaissance	Nmap NSE scan	Rule 100012
T1190	Exploit Public-Facing App	Initial Access	SQLi auth bypass	Rule 31106 / 31333
T1059.007	JavaScript Execution	Execution	XSS injection	Rule 31105 / 100015
T1110	Brute Force	Credential Access	Hydra login attack	Rule 100010
T1078	Valid Accounts	Defense Evasion	Admin endpoint access	Rule 100014
T1083	File & Dir Discovery	Discovery	Path traversal + .git probe	Rule 31333
T1548.003	Sudo Abuse	Privilege Escalation	sudo -i on victim VM	Rule 5402
T1562.001	Disable Security Tools	Defense Evasion	Wazuh agent stopped on reboot	Rule 506

9. Findings & Recommendations

9.1 Findings

ID	Severity	Title	Description
F001	CRITICAL	SQL Injection Auth Bypass	No server-side input sanitization on login. Admin JWT obtained with single command.
F002	HIGH	No Brute Force Protection	No rate limiting, lockout, or CAPTCHA. Hydra ran 102 attempts in under 5 seconds.
F003	HIGH	Admin API Unrestricted	/rest/admin/* returns full config to any requester with no authentication check.
F004	MEDIUM	POST Body Not Logged	Nginx logs paths but not POST body - SQLi payloads invisible in access logs.
F005	LOW	Git Repo Exposed	/.git/HEAD returns HTTP 200 - potential source code exposure vector.

9.2 Recommendations

1. Enable ModSecurity WAF in production - OWASP CRS blocked 100% of tested attacks with zero tuning. Highest-impact single control available.
 2. Implement Nginx rate limiting - add limit_req_zone to complement WAF and prevent brute force even if WAF rules are bypassed.
 3. Enable POST body logging - configure Nginx log format to capture request body snippets for forensic visibility of SQLi attacks.
 4. Restrict admin API - apply IP allowlisting or JWT authentication to /rest/admin/* at Nginx level before reaching application.
 5. Configure Wazuh active response - automatic iptables block when Rule 100013 fires (10+ WAF blocks from same IP in 60 seconds).
 6. Expand custom rules - add Burp Suite User-Agent, web shell patterns, and JWT manipulation detection.
 7. Block sensitive paths at Nginx level - deny /.git/, /.env, /admin using deny rules in Nginx configuration.
-

10. Next Steps - Lab Roadmap

Phase 3 - Advanced Attack Techniques

- Burp Suite manual exploitation - IDOR, broken access control, business logic flaws
- WAF bypass techniques - encoding, case variation, chunked transfer encoding
- SSRF (Server-Side Request Forgery) against internal services
- JWT token manipulation - algorithm confusion and signature bypass
- File upload vulnerabilities and web shell delivery through Juice Shop

Phase 4 - Detection Engineering

- Wazuh active response - automatic IP blocking on attack threshold breach
- Custom decoders for structured Juice Shop JSON application logs
- SIEM dashboard - attack timeline, top attacker IPs, rule fire frequency charts
- Threat hunting queries - proactive IOC search in historical data
- Sigma rule development - portable detection rules exportable to other SIEMs

Phase 5 - Incident Response Simulation

- Full attack chain simulation: recon to exploit to persistence to exfiltration
- IR runbook development for each attack type documented in this report
- Purple team exercise - attacker and defender working simultaneously
- Forensic artifact collection and chain of custody documentation

Phase 6 - Compliance & Hardening

- CIS Ubuntu 24.04 Benchmark - review Wazuh SCA scan results and remediate findings
 - PCI DSS controls mapping using Wazuh built-in compliance dashboard
 - NIST 800-53 control implementation review
 - Vulnerability scanning with OpenVAS against victim VM
-

Appendix A - Configuration Reference

A.1 Systemd Services - Victim VM

Service	Status	Enabled	Purpose
juiceshop.service	active (running)	yes	OWASP Juice Shop Node.js app on port 3000
nginx.service	active (running)	yes	Reverse proxy + ModSecurity WAF on port 80
wazuh-agent.service	active (running)	yes	Security telemetry agent - forwards logs to SIEM
systemd-timesyncd	active (running)	yes	NTP time synchronization

A.2 Wazuh Manager Services - SIEM VM

Service	Status	Purpose
wazuh-manager	active (running)	Core analysis and correlation engine
wazuh-indexer	active (running)	OpenSearch alert storage and indexing
wazuh-dashboard	active (running)	Web interface on HTTPS port 443
filebeat	active (running)	Log forwarding from manager to indexer

A.3 Snapshot History - Victim VM

Snapshot Name	State	Purpose
Fase1-SinWAF-JuiceShop+Nginx+WazuhAgent-OK	Saved	Baseline before WAF installation
Fase2-WAF-ModSecurity-CRS-Wazuh-Validado	Saved	Full stack with WAF active and validated

A.4 Wazuh Agent Log Sources (ossec.conf)

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/juiceshop_access.log</location>
</localfile>
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/juiceshop_error.log</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>
```

```
<localfile>  
  <log_format>journald</log_format>  
  <location>journald</location>  
</localfile>
```